

EventCamArucoDetector

Detecting ArUco markers from raw event-camera streams

This document walks through the full pipeline used to find ArUco markers in the asynchronous output of an event camera. It covers every stage end-to-end: data layout, the multi-window sliding-tick scheme, quad detection, perspective rectification, scanline transition decoding, dictionary lookup, the optional marker-ID filter, result merging across runs, and the tabbed viewer that visualises the result tables.

EventCamArucoDetector · end-to-end pipeline

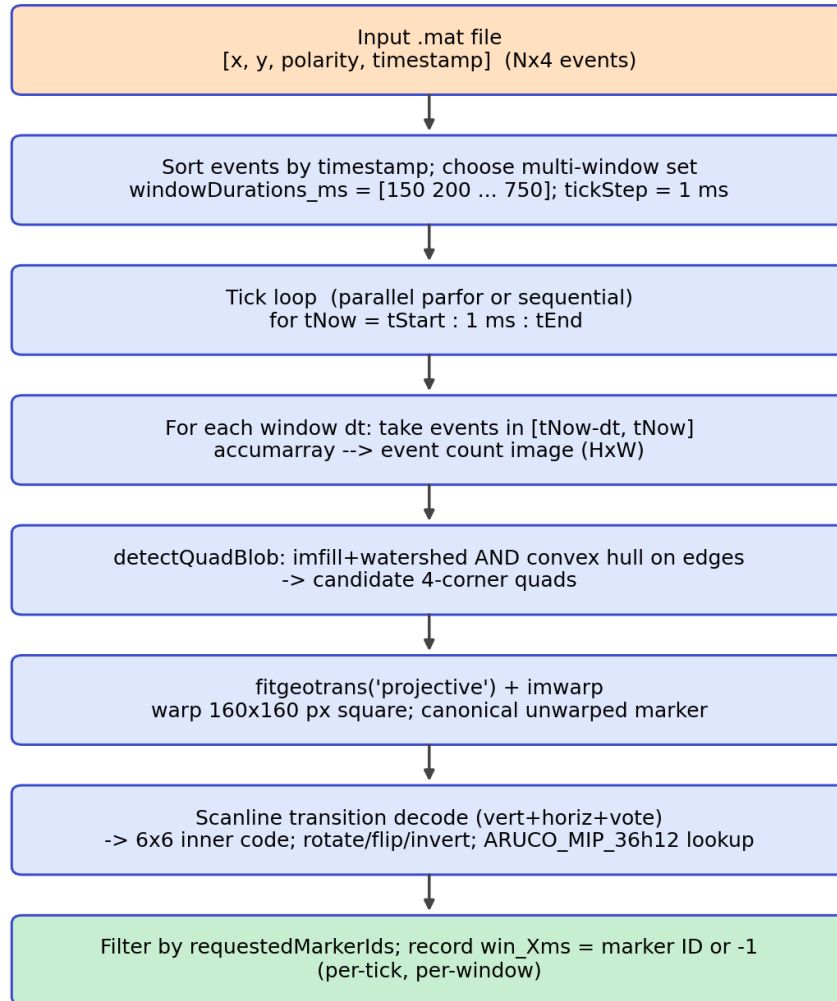


Figure 1 - End-to-end pipeline. Each box maps directly to a code unit in the repository.

1. Why event cameras need a different detector

A standard ArUco detector expects a global-shutter intensity image: find an external contour, fit a quadrilateral, threshold, decode. Event cameras don't produce frames at all. They emit an asynchronous stream of 4-tuples - (**x, y, polarity, timestamp**) - one per pixel whenever the log-intensity at that pixel changes by more than a contrast threshold. There is no concept of "the picture at time t"; you only ever see what changed.

Static markers therefore produce *no* events. We rely on either the marker moving in the scene or the camera moving relative to it. When that happens, the high-contrast border of the marker triggers events at every crossing, which gives us edges to work with - but no fill, no greyscale and no guaranteed continuity.

1.1 Input data layout

Each dataset is stored as a MATLAB .mat file containing a single Nx4 array called events:

```
events(:,1) = x      (column, 0-based)
events(:,2) = y      (row, 0-based)
events(:,3) = polarity (+1 on, 0/-1 off; ignored by the detector)
events(:,4) = t      (timestamp in microseconds)
```

The DVS we used produces events on a 320x240 sensor (sensorSize = [240, 320]). At typical motion speeds this is around 1-5 million events per recording, sparse in space and dense in time.

2. Multi-window sliding-tick scheme

We do *not* try to track individual events. Instead we treat the stream as a piecewise time-integration problem. At every **tick** - by default every 1 ms - we ask the same question for several different lookback durations:

- How many events occurred in the window [tNow - dt, tNow]?
- Treat those events as a static binary image.
- Run the rest of the pipeline on that image.

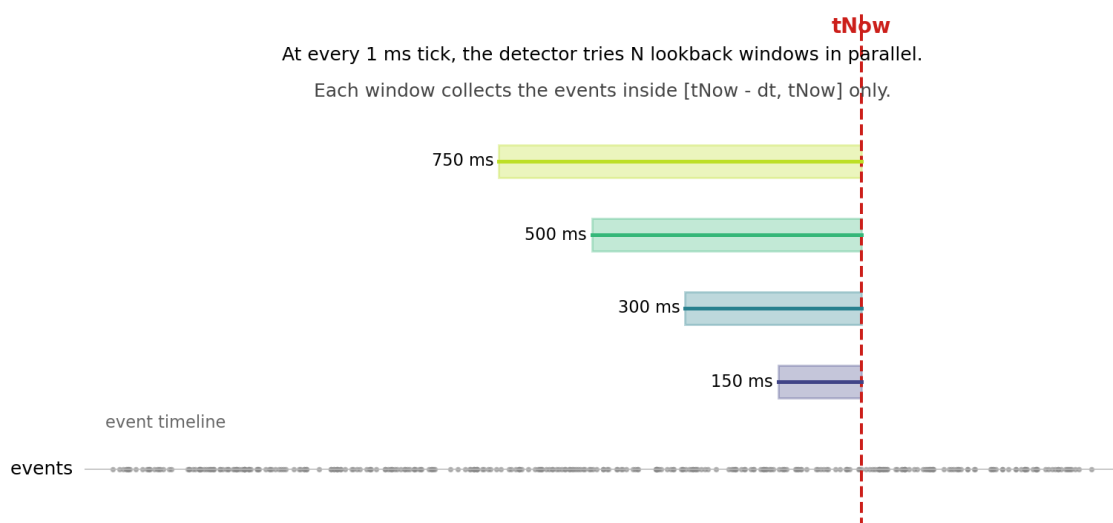


Figure 2 - At each 1 ms tick we run several windows in parallel. The set of windows is configured by params.windowDurations_ms.

Why several windows? Because the best integration time depends on the motion: a slow rotation needs hundreds of milliseconds to accumulate a clean edge, but for fast translation the same window

will smear the marker into a blur. By trying multiple windows at the same tick and recording which (if any) succeeded, we get a robust detector and we can later analyse which window is best for which motion. That analysis is exactly what the result viewer exposes.

2.1 Where do we start and stop?

Setting up the tick loop is one of the few places this code is fussy. We have to skip the first $\max(\text{windowDurations})$ of the recording, because before that point no window has enough history. The relevant lines from `detectAruco.m`:

```
tMin = evT(1);
tMax = evT(end);
tStart = tMin + max(windowDurations_us);
tEnd = tMax;
numTicks = floor((tEnd - tStart) / tickStep_us) + 1;
```

Note that running the same recording with a different window set therefore produces a different number of ticks - if we add a 750 ms window we lose the first 750 ms of the recording. That is why `mergeResults` has to be careful about timelines.

3. From events to a binary image

Inside the window we have a list of (x, y) pixels that fired at least once. Two binary-search calls extract the right slice of the sorted `evT` array; `accumarray` then builds the per-pixel count image in one vectorised step:

```
iStart = bsearchLeft(evT, tNow - dt);
iEnd = bsearchRight(evT, tNow);
wX = evX(iStart:iEnd); wY = evY(iStart:iEnd);
countImg = accumarray([wY, wX], 1, [H, W]);
activeMask = countImg > 0;
```

`activeMask` is what the quad detector consumes. `countImg` survives for later: when we want to *decode* a candidate marker we need greyscale-like intensity, so we use `uint8(countImg / max(countImg) * 255)` instead of the binary mask.

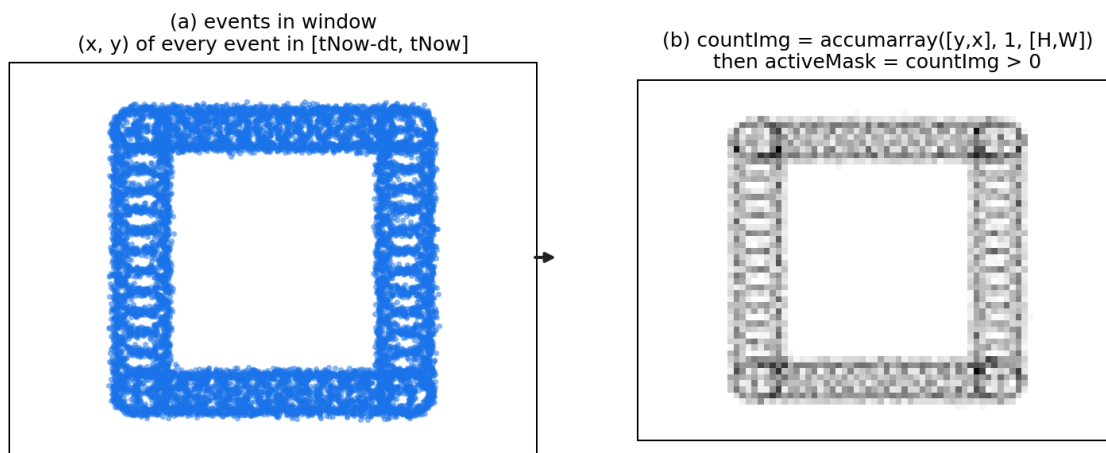


Figure 3 - Left: the raw (x, y) of every event in the window. Right: the same events accumulated into an HxW count image.

3.1 Sub-threshold rejection

If a window contains fewer than 10 events we skip it altogether - any quad we could fit would be noise. This is the cheapest early-exit in the loop.

4. Quad detection - two methods, then dedup

Event cameras give us only the marker's edges, never its fill. Sometimes the edges form a closed loop, sometimes there is a missing segment because the camera was momentarily still. We therefore run two complementary detectors on every frame and merge their hits.

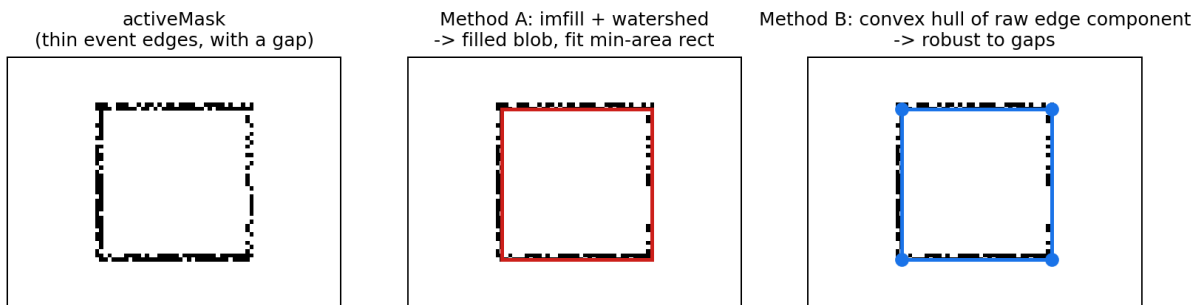


Figure 4 - Method A fills closed boundaries and fits a minimum-area rectangle to the filled blob. Method B takes the convex hull of the raw edge component, which survives a missing side.

Method A - fill + watershed (closed-edge markers)

- `imfill(activeMask, 'holes')` closes the marker interior.
- `bwareaopen(bw, 100)` drops noise blobs.
- Distance transform + extended-maxima + watershed splits two markers that have touched.
- `findQuadCandidates` fits a minimum-area rectangle to the convex hull of each surviving blob and accepts it if area, aspect, and rectangularity all pass.

Method B - convex hull on raw edges (markers with gaps)

- Run `bwconncomp` directly on `activeMask` with 4-connectivity (so two pixels need to share a side, not just a corner).
- For each component, take the convex hull and fit a minimum-area rectangle as before.
- Because we work on the convex hull, a missing side does not invalidate the candidate.

Deduplication

Both methods often return the same physical marker. `deduplicateQuads` walks the combined list and drops any quad whose centre is within half a side-length of an earlier one. The first-found wins.

4.1 The filters that actually do the work

Inside `findQuadCandidates` three predicates decide whether a region looks like an ArUco frame:

- **Area filter** - the *convex-hull* area must be between `params.minArea` (default 625 px) and `params.maxArea` (default 40% of the image).
- **Rectangularity** = `convexArea / minRectArea` must exceed 0.6. Genuine squares score close to 1; a noisy blob with a long tail scores much lower.
- **Aspect ratio** of the minimum-area rectangle must be below 3.0. Elongated rectangles are not markers.

5. Perspective rectification

Each surviving quad gives us four image-space corners. We need to flatten that quad into a canonical 160 x 160 px square so the rest of the decoder can assume axis-aligned cells. We do this with a

projective homography:

```
tform = fitgeotrans(srcCorners, dstCorners+1, 'projective');
warpedImg = imwarp(countU8, tform, 'OutputView', ...);
```

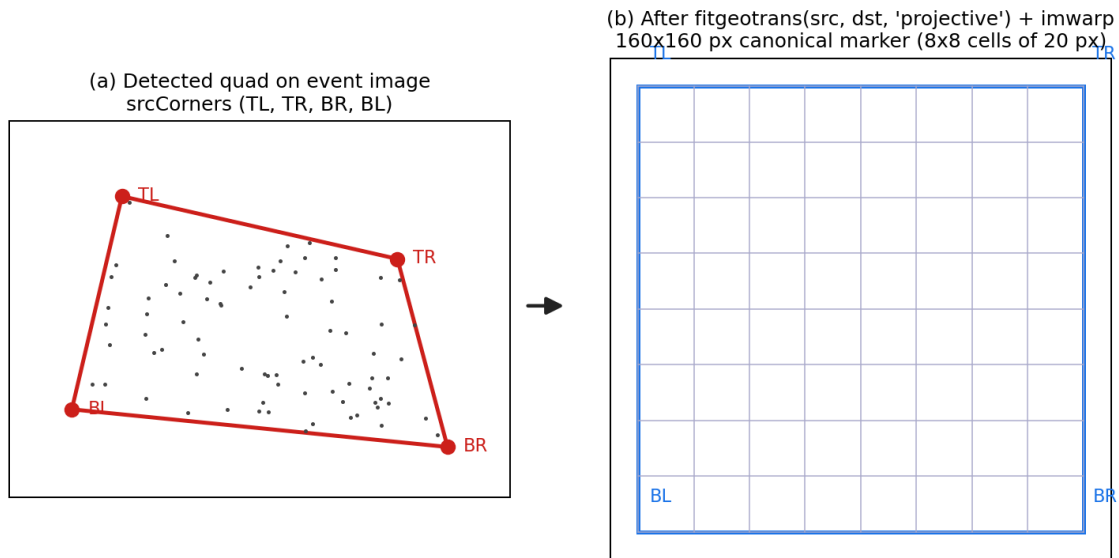


Figure 5 - A distorted detected quad is mapped onto a fixed 8x8 grid of 20-pixel cells (160x160 px).

5.1 Corner ordering

fitgeotrans only behaves correctly if the source and destination corners are in matching order. orderCornersForUnwarp_local sorts the quad by angle around its centroid, then rotates the list so the corner with the smallest x+y is first (top-left), and finally flips the order if the cross product disagrees with the canonical (TL, TR, BR, BL) winding. This makes the unwarp deterministic regardless of which corner the detector returned first.

6. Scanline transition decoding

Now we have a 160x160 image of a marker. A normal photo would let us threshold each cell directly to read 0/1. Event-derived images don't have a meaningful interior - the only signal is at the cell boundaries, where the polarity flips. We decode by following the scanlines and toggling colour at every boundary that crosses a learned threshold.

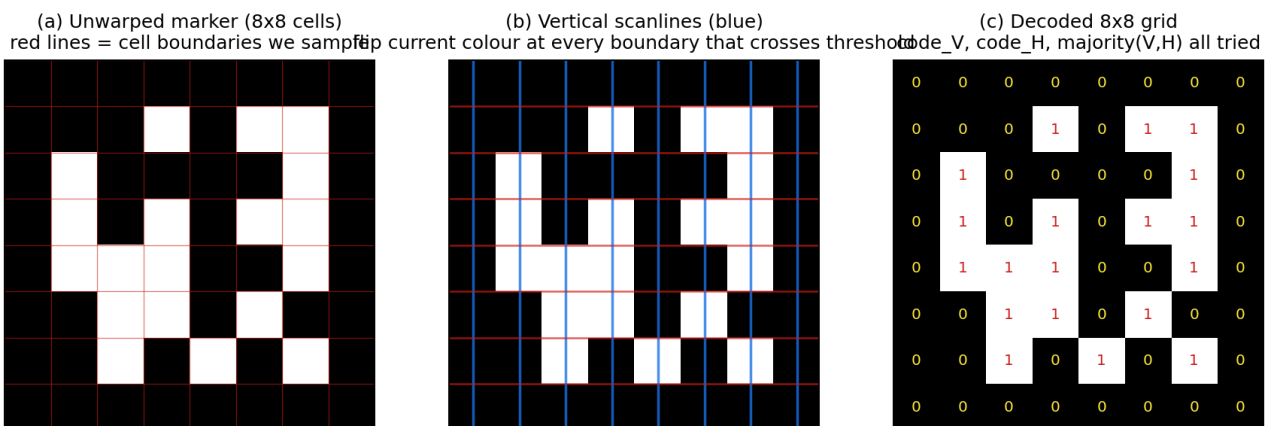


Figure 6 - Cell boundaries (red) are where event intensity peaks. Vertical and horizontal scanlines (blue) sample column / row centres and flip the current colour at each boundary that exceeds the auto-tuned threshold.

6.1 Boundary intensity, then Otsu threshold

For every cell boundary we sample the mean intensity in a narrow 11-pixel band (parameter `boundaryHalfWidth = 5`). The full set of boundary samples is then thresholded with `graythresh` (Otsu's method) to find the per-marker `transThresh`. This adapts to whatever brightness the current accumulated count image happens to have.

6.2 Three candidate codes, six geometric variants

Reading the marker as a single vertical scan or a single horizontal scan can fail when one direction has a weak boundary; we therefore build *three* candidate 8x8 codes:

- `codeV` - scan each column top-to-bottom, flipping colour at every horizontal boundary above threshold.
- `codeH` - scan each row left-to-right, flipping colour at every vertical boundary above threshold.
- `majority` - bit-wise majority of `codeV` and `codeH`, which heals individual misreads.

Decoder tries 48 variants per candidate before giving up

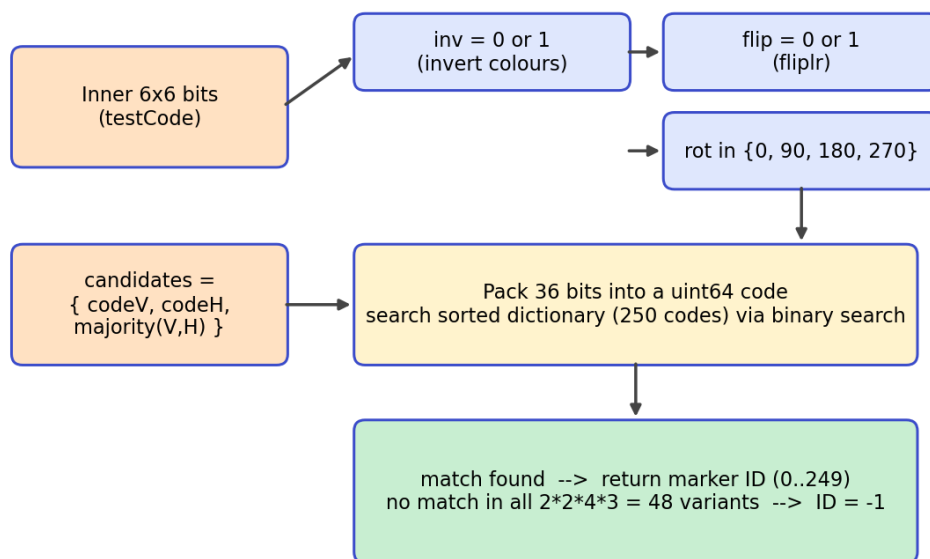


Figure 7 - For each of the three candidate codes, the decoder tries all $2 \times 2 \times 4 = 16$ combinations of invert, horizontal flip, and 90-degree rotation before giving up. Including the three candidates that is 48 lookups per quad, each a single binary search.

6.3 Inner code and dictionary lookup

After dropping the outer black border (rows/cols 1 and 8) we have a 6x6 grid of bits. We pack those 36 bits into a `uint64` with the dictionary's canonical bit order and binary-search the `ARUCO_MIP_36h12` dictionary (250 codes), which is pre-sorted at startup. A hit returns the marker ID (0 to 249); a miss makes us try the next variant.

7. The requestedMarkerIds filter

Most experiments only care about *one* marker - the one printed in the recording. We added `params.requestedMarkerIds` so that decodes outside that set are treated as no-detection:

```

params.requestedMarkerIds = 3;           % only accept marker ID 3
params.requestedMarkerIds = [3 7 12];    % accept any of these
params.requestedMarkerIds = [];          % accept any decoded ID
  
```

Concretely, inside the quad loop the check changes from

```
if mid >= 0
    bestID = mid; break;
end
```

to

```
if mid >= 0 && (isempty(requestedMarkerIds) || ...
    any(requestedMarkerIds == int32(mid)))
    bestID = mid; break;
end
```

The decoded ID is still stored per window (in the win_Xms columns) so that the viewer can slice the results by marker after the fact - even when several IDs are recorded inside the same run, you can still ask "how often did marker 7 show up?"

8. Output schema

Every detection run writes a struct with these fields:

field	shape	meaning
tNow_us	Nx1 double	timestamp of each tick (microseconds)
anyDetected	Nx1 double	1 if any window detected this tick, else 0
win_Xms	Nx1 double	decoded marker ID for window X ms (-1 = none)
windowDurations_ms	1xW double	the window set used
detectionsPerWindow	1xW double	total hits per window across all ticks
requestedMarkerIds	1xK double	filter that was applied ([] = any)
attemptedPerWindow	1xW double	(after merge) how many ticks each window was actually run on

8.1 Merging multiple runs over the same input

Running detection twice on the same recording with different window sets gives us two result files with *different* tick counts (because tStart depends on max(windowDurations)). mergeResults joins them on the union of timestamps, fills missing entries with -1, and records attemptedPerWindow so we can report an honest per-window rate.

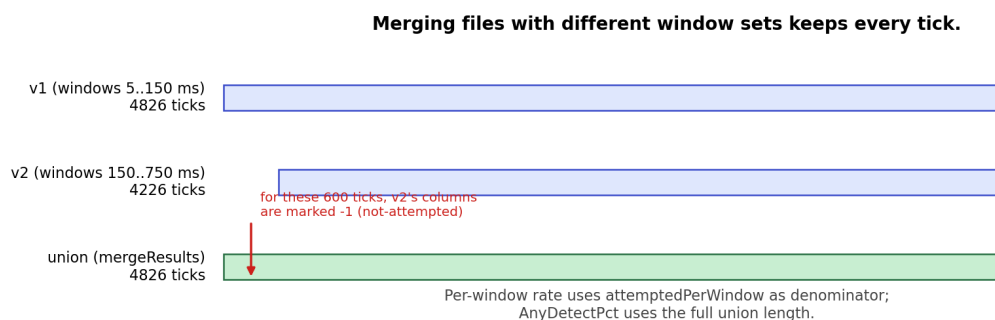


Figure 8 - Union-of-timestamps merge. v1 covers the full recording but only with small windows; v2 skips the first 600 ticks but adds large windows. The merged file keeps every tick and is honest about which window was attempted where.

There are two natural denominators after a merge, and viewAllResults uses both:

- **AnyDetectPct** = #ticks with any successful window / *union length*. This is the right number for "how often did we see the marker at all?"
- **BestRatePct** = #detections of that window / *attemptedPerWindow* of that window. This is the right number for "when this window was tried, how often did it succeed?"

Because the denominators differ, you can legitimately see AnyDetectPct = 95.3% next to a 500 ms window at 100% - the first 600 ticks were only covered by smaller windows in v1, which missed some, while the 500 ms window from v2 nailed everything it was actually tried on.

9. The result viewer

viewAllResults('Data') opens a single window with one summary tab and one detail tab per dataset. A marker-filter dropdown lets you recompute every tab for a specific decoded ID without re-running detection. Three save buttons export the current tab, the current window, or all tabs as a multi-page PDF.

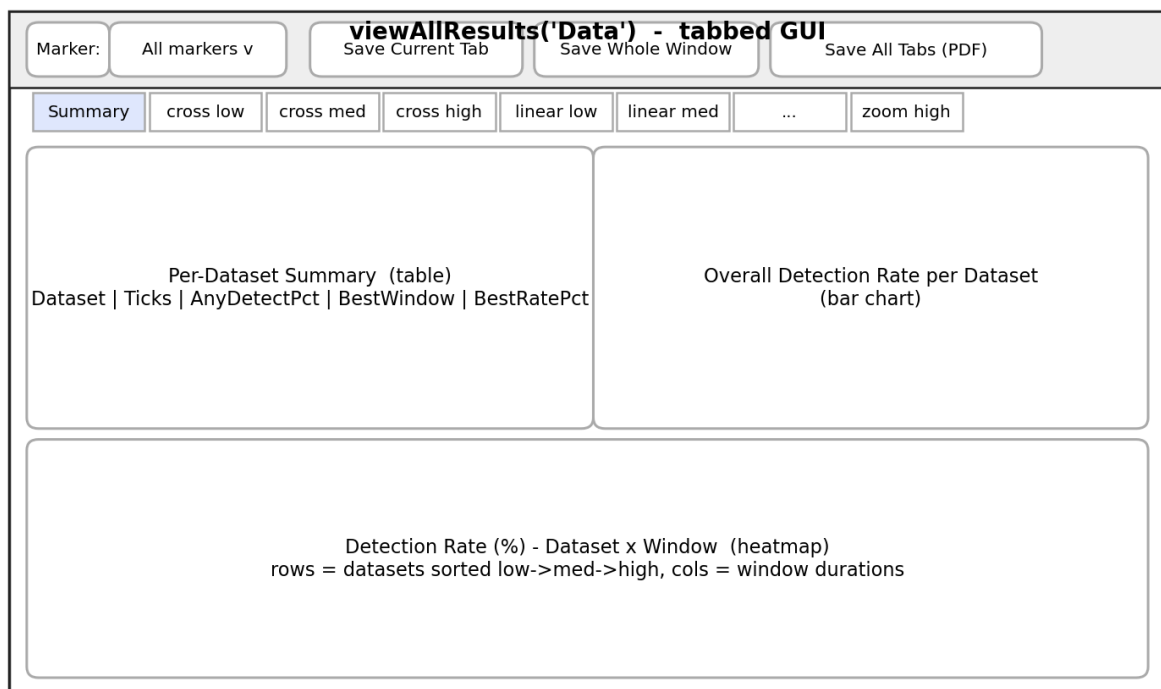


Figure 9 - The summary tab. The dropdown switches the view between "all markers" and any specific ID found in the data.

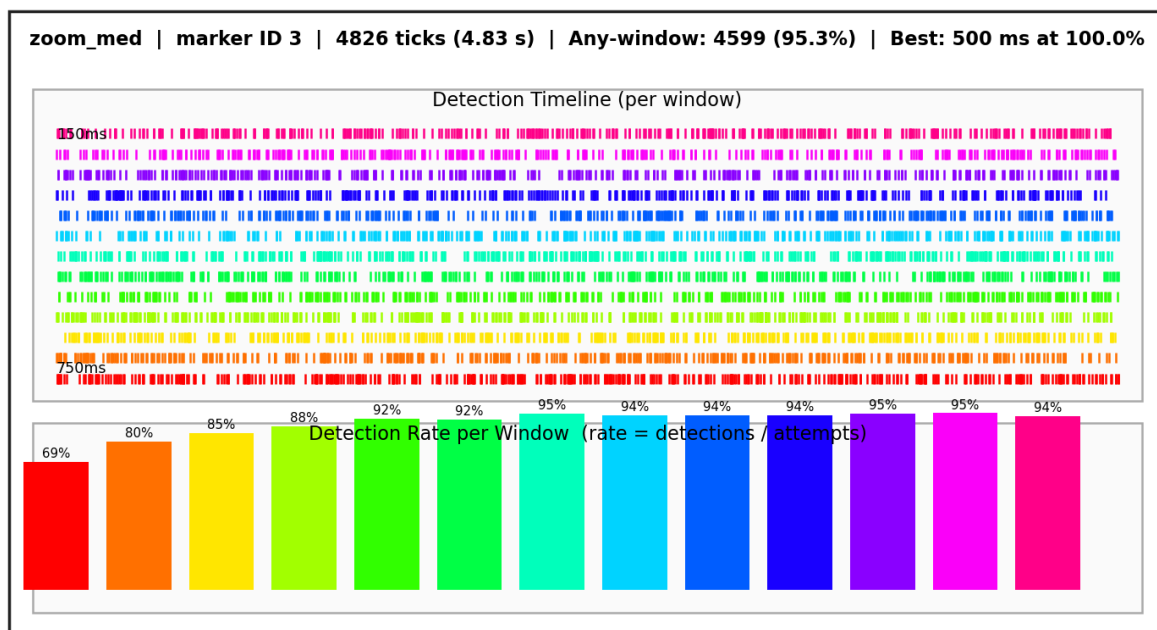


Figure 10 - A per-dataset detail tab. The raster shows when each window fired; the bar chart shows the per-window success rate using attemptedPerWindow as denominator.

10. Parallelism and performance notes

- The tick loop is **embarrassingly parallel**; each tick is independent. With Parallel Computing Toolbox and `params.useParallel = true` we use `parfor` and a `parallel.pool.DataQueue` for live progress.
- Without the toolbox (or when `params.useParallel = false`) the same loop runs sequentially with a 1%-resolution progress print.
- Dictionary lookup is $O(\log 250)$ per quad per variant - `buildDictionaryArrays` sorts the codes once so the worker copies are immutable.
- `accumarray` is the only big-O hotspot inside the inner loop and is fully vectorised.

10.1 What's the bottleneck?

Empirically the dominant cost is imwarp, not decoding. If a recording has many false-positive quads, we pay one perspective warp per quad per window per tick. The `minRectangularity` and `maxAspect` filters in `findQuadCandidates` are the main lever for keeping that in check.

11. Where the bodies are buried

Things worth remembering when you read or modify the code:

- `events(:,3)` (polarity) is ignored. The detector only uses positions and timestamps.
- Marker decoding tries 48 variants per candidate. If you speed-tune this, that is the loop to look at.
- `tStart` depends on the *largest* requested window. Changing the window set shifts the start, so tick counts change between runs and have to be merged on the union of timestamps.
- With a non-empty `requestedMarkerIds`, decodes outside the set are stored as -1 in the relevant cell. This is silent: nothing in the result file tells you "by the way, I rejected three valid marker-9 hits here". If that matters, log them in `detectAruco`.

- Merging an unfiltered v1 with a filtered v2 produces a file whose semantics differ across windows. The viewer's marker dropdown will surface IDs that exist only in some windows. Keep the filter consistent across runs you intend to merge.

12. File map

file	responsibility
main.m	configure inputs / parameters, drive the per-dataset loop, kick off merge + viewer
detectAruco.m	tick loop, event accumulation, decoder, the parfor/sequential split
detectQuadBlob.m	Method A (fill+watershed) + Method B (convex hull on edges) + dedup
findQuadCandidates.m	area / rectangularity / aspect filters, minimum-area rect fit, corner ordering
Utils/mergeResults.m	union-timeline merge of two or more result files
Utils/mergeAllResults.m	batch-merge every v1/v2 pair under Data/
viewAllResults.m	tabbed GUI, marker-filter dropdown, PDF exports
analyzeResults.m	single-result deep-dive: raster, bars, heatmap, gap distribution, rolling rate